

tf.compat.v1.distribute.experimental.MultiWorkerMirroredStrategy

Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/distribute/experimental/MultiWorkerMirroredStrategy

A distribution strategy for synchronous training on multiple workers.

Function Signatures

```
tf.compat.v1.distribute.experimental.MultiWorkerMirroredStrategy(  
    communication=tf.distribute.experimental.CollectiveCommunication.AUTO,  
    cluster_resolver=None )
```

Code Examples

```
tf.compat.v1.distribute.experimental.MultiWorkerMirroredStrategy(  
    communication=tf.distribute.experimental.CollectiveCommunication.AUTO,  
    cluster_resolver=None  
)
```

```
tf.compat.v1.distribute.experimental.MultiWorkerMirroredStrategy(  
communication=tf.distribute.experimental.CollectiveCommunication.AUTO,  
cluster_resolver=None  
)
```

```
tf.distribute.experimental.CollectiveCommunication.AUTO
```

```
TF_CONFIG = '{"cluster": {"worker": ["localhost:12345",  
"localhost:23456"]}, "task": {"type": "worker", "index": 0} }'
```

```
TF_CONFIG = '{"cluster": {"worker": ["localhost:12345",  
"localhost:23456"]}, "task": {"type": "worker", "index": 0} }'
```

```
strategy = tf.distribute.MultiWorkerMirroredStrategy()  
with strategy.scope():  
model = tf.keras.Sequential([  
tf.keras.layers.Dense(2, input_shape=(5,)),  
)  
optimizer = tf.keras.optimizers.SGD(learning_rate=0.1)  
def dataset_fn(ctx):  
x = np.random.random((2, 5)).astype(np.float32)  
y = np.random.randint(2, size=(2, 1))  
dataset = tf.data.Dataset.from_tensor_slices((x, y))  
return dataset.repeat().batch(1, drop_remainder=True)  
dist_dataset =  
strategy.distribute_datasets_from_function(dataset_fn)  
model.compile()  
model.fit(dist_dataset)
```

```

strategy = tf.distribute.MultiWorkerMirroredStrategy()
with strategy.scope():
    model = tf.keras.Sequential([
        tf.keras.layers.Dense(2, input_shape=(5,)),
    ])
    optimizer = tf.keras.optimizers.SGD(learning_rate=0.1)
    def dataset_fn(ctx):
        x = np.random.random((2, 5)).astype(np.float32)
        y = np.random.randint(2, size=(2, 1))
        dataset = tf.data.Dataset.from_tensor_slices((x, y))
        return dataset.repeat().batch(1, drop_remainder=True)
    dist_dataset =
    strategy.distribute_datasets_from_function(dataset_fn)
    model.compile()
    model.fit(dist_dataset)

```

```

@tf.function
def train_step(iterator):
    def step_fn(inputs):
        features, labels = inputs
        with tf.GradientTape() as tape:
            logits = model(features, training=True)
            loss = tf.keras.losses.sparse_categorical_crossentropy(
                labels, logits)
            grads = tape.gradient(loss, model.trainable_variables)
            optimizer.apply_gradients(zip(grads, model.trainable_variables))
        strategy.run(step_fn, args=(next(iterator),))
    for _ in range(NUM_STEP):
        train_step(iterator)

```

Documentation

A distribution strategy for synchronous training on multiple workers.

Inherits From: Strategy

This strategy implements synchronous distributed training across multiple workers, each with potentially multiple GPUs. Similar to `tf.distribute.MirroredStrategy`, it replicates all variables and computations to each local device. The difference is that it uses a distributed collective implementation (e.g. all-reduce), so that multiple workers can work together.

You need to launch your program on each worker and configure `cluster_resolver` correctly. For example, if you are using `tf.distribute.cluster_resolver.TFConfigClusterResolver`, each worker needs to have its corresponding `task_type` and `task_id` set in the `TF_CONFIG` environment variable. An example `TF_CONFIG` on worker-0 of a two worker cluster is:

Your program runs on each worker as-is. Note that collectives require each worker to participate. All `tf.distribute` and non `tf.distribute` API may use collectives internally, e.g. checkpointing and saving since reading a `tf.Variable` with `tf.VariableSynchronization.ON_READ` all-reduces the value. Therefore it's recommended to run exactly the same program on each worker. Dispatching based on `task_type` or `task_id` of the worker is error-prone.

`cluster_resolver.num_accelerators()` determines the number of GPUs the strategy uses. If it's zero, the strategy uses the CPU. All workers need to use the same number of devices, otherwise the behavior is undefined.

This strategy is not intended for TPU. Use `tf.distribute.TPUStrategy` instead.

After setting up `TF_CONFIG`, using this strategy is similar to using `tf.distribute.MirroredStrategy` and `tf.distribute.TPUStrategy`.

You can also write your own training loop:

See

Multi-worker training with Keras
for a detailed tutorial.

Saving

You need to save and checkpoint on all workers instead of just one. This is because variables whose `synchronization=ON_READ` triggers aggregation during saving. It's recommended to save to a different path on each worker to avoid race conditions. Each worker saves the same thing. See Multi-worker training with Keras tutorial for examples.

Known Issues

- `tf.distribute.cluster_resolver.TFConfigClusterResolver` does not return the

correct number of accelerators. The strategy uses all available GPUs if `cluster_resolver` is `tf.distribute.cluster_resolver.TFConfigClusterResolver` or `None`.

- In eager mode, the strategy needs to be created before calling any other

Tensorflow API.

Attributes

In general, when using a multi-worker `tf.distribute` strategy such as

`tf.distribute.experimental.MultiWorkerMirroredStrategy` or `tf.distribute.TPUStrategy()`, there is a `tf.distribute.cluster_resolver.ClusterResolver` associated with the strategy used, and such an instance is returned by this property.

Strategies that intend to have an associated `tf.distribute.cluster_resolver.ClusterResolver` must set the relevant attribute, or override this property; otherwise, `None` is returned by default. Those strategies should also provide information regarding what is returned by this property.

Single-worker strategies usually do not have a `tf.distribute.cluster_resolver.ClusterResolver`, and in those cases this property will return `None`.

The `tf.distribute.cluster_resolver.ClusterResolver` may be useful when the user needs to access information such as the cluster spec, task type or task id. For example,

For more information, please see `tf.distribute.cluster_resolver.ClusterResolver`'s API docstring.

Methods

`## distribute_datasets_from_function`

[View source](#)

Distributes `tf.data.Dataset` instances created by calls to `dataset_fn`.

The argument `dataset_fn` that users pass in is an input function that has a

`tf.distribute.InputContext` argument and returns a `tf.data.Dataset` instance. It is expected that the returned dataset from `dataset_fn` is already batched by per-replica batch size (i.e. global batch size divided by the number of replicas in sync) and sharded.

`tf.distribute.Strategy.distribute_datasets_from_function` does not batch or shard the `tf.data.Dataset` instance returned from the input function. `dataset_fn` will be called on the CPU device of each of the workers and each generates a dataset where every replica on that worker will dequeue one batch of inputs (i.e. if a worker has two replicas, two batches will be dequeued from the Dataset every step).

This method can be used for several purposes. First, it allows you to specify your own batching and sharding logic. (In contrast, `tf.distribute.experimental_distribute_dataset` does batching and sharding for you.) For example, where `experimental_distribute_dataset` is unable to shard the input files, this method might be used to manually shard the dataset (avoiding the slow fallback behavior in `experimental_distribute_dataset`). In cases where the dataset is infinite, this sharding can be done by creating dataset replicas that differ only in their random seed.

The `dataset_fn` should take an `tf.distribute.InputContext` instance where information about batching and input replication can be accessed.

You can use `element_spec` property of the `tf.distribute.DistributedDataset` returned by this API to query the `tf.TypeSpec` of the elements returned by the iterator. This can be used to set the `input_signature` property of a `tf.function`. Follow `tf.distribute.DistributedDataset.element_spec` to see an example.

For a tutorial on more usage and properties of this method, refer to the tutorial on distributed input).

If you are interested in last partial batch handling, read this section.

experimental_distribute_dataset

[View source](#)

Creates `tf.distribute.DistributedDataset` from `tf.data.Dataset`.

The returned `tf.distribute.DistributedDataset` can be iterated over similar to regular datasets.

NOTE: The user cannot add any more transformations to a `tf.distribute.DistributedDataset`. You can only create an iterator or examine the `tf.TypeSpec` of the data generated by it. See API docs of `tf.distribute.DistributedDataset` to learn more.